

Claude's plan

Calls, the ringtone, timing and the measuring behind them

For Mutalib, in plain words. These are the tasks underneath the screens: how Thrum notices a call, how the ringtone and the vibration stay together, how fast a new song starts, and the measuring that proves each one. Antigravity's half (wording, light mode, glass, the dial, the bars) is in `antigravity-2026-10-07.md` beside this file. The order is below; every task ends with something you can hear, feel or read on your own phone.

The order: Task 31 and Task 32 together (your call), then 37, 34, 35 and 38. Task 33 waits for Task 32's answer.

Status, 7 October, afternoon: waiting for your start. You asked for this plan as a PDF and said you'll say when to start. Before that, Task 31's code was written on the branch `claude/task-31-ringtone` (commits `23f75c4` and `7b6ee40`): the tests pass, lint is clean and it installs on the emulator. It hasn't been tried on a phone yet and isn't merged. Nothing else has started.

How each task runs: its own branch from `ui-final-screens` (`claude/task-31-ringtone` and so on), `check.ps1` and lint before every commit, commit and push after every meaningful step, no AI attribution anywhere. It's merged back into `ui-final-screens` once the proof on your phone is in. At the end, a plain walkthrough: the problem, how it was solved, the one idea that made it work, and what went wrong.

Task 31: One ringtone action (now)

In plain words. One button, "Set as ringtone", does everything: your phone rings with the song and Thrum vibrates to the same song. The ringtone is the song's first 45 seconds, so sound and vibration start over together. Thrum remembers the ringtone you had, and Settings can put it back.

Decisions it follows. Mutalib, 7 October: one action; ringtone = first 45 seconds; remember the old ringtone. On the two follow-up questions ("don't worry about the options") I went with my recommendations, and both are easy to undo: Thrum plays in Ring mode only when the ringtone is the Thrum song, and "Go back to my old ringtone" also stops Thrum's call vibration.

Steps

1. `RingtoneRules.kt` (new, plain Kotlin, tested on the PC): whether two ringtone addresses are the same, whether the ringtone is the song calls vibrate to, when to remember the old one, when a song needs cutting, the 150 ms fade at the end of a cut, and how far to skip ahead on a late start.
2. `RingtoneClip.kt` (new): makes the 45-second ringtone on the phone. It reads the song's sound with Android's own decoder, keeps the first 45 seconds (stereo, or mono for anything wider), fades the last 150 ms, and writes an AAC `.m4a` file with Android's own encoder and `MediaMuxer`. No FFmpeg and nothing downloaded: it uses the converters every Android phone has. Works the same for songs, picked files and videos. A song of 45.5 seconds or less is used as it is.
3. `Ringtone.kt`: the clip goes into the phone's Ringtones/Thrum folder as "`<song>` (Thrum)" and is reused if it's already there. Before changing anything it notes the user's own ringtone once (never one of Thrum's),

then sets the clip, reads the setting back as it does today, and records which ringtone it set and for which song. New: `isTheSong` (asked at call time and on Home) and `restoreOld`.

4. `Store.kt`: three new values (the ringtone Thrum set, the song it was for, the user's old ringtone) and `disarm()`. The "Also when the ringer is on" switch goes.
5. `RowActions.kt`: one flow for every entry point. Already allowed → set it; not allowed → the existing explanation sheet; "Just the vibration" still works, and then Thrum plays on vibrate only.
6. `NotifService.kt`: in Ring mode, plays only when `isTheSong`; otherwise it stands down and logs why. When it plays, it starts as far into the rhythm as it was late (`catchUpMs`), and keeps re-asserting from that corrected start. The call log notes how far it caught up, for measuring.
7. Screens: Home's card says "Ringtone"; "Change" picks a song and sets it as the ringtone; in Ring mode with a different ringtone, Home says so and offers "Set as ringtone". The player's "Use for calls" becomes "Set as ringtone". Settings: "Ringtone" row, plus "Go back to my old ringtone" with its name once there is one. Imported haptics (no sound) keep a separate item, "Vibrate for calls", which says plainly that it plays on vibrate only.
8. Words: every "song for calls" sentence rewritten to match, humanizer-gated. The onboarding call page too.
9. `PROFILE.md` §4 items 3, 11 and 17 updated to say all of this.

Tests on the PC: the rules in step 1, and the Home verdict table, with its ring-mode rows now about the ringtone instead of the switch.

Done when, on the Pixel

- One "Set as ringtone" on Home, the player and the : menu.
- `adb shell settings get system ringtone` reads back the clip, and the clip pulled off the phone is 45.0 s long (checked with `ffprobe`).
- "Go back" restores the noted ringtone, checked the same way.
- On real calls in Ring mode, Mutalib's hand can't feel the vibration start late or jump back. `dumpsys vibrator_manager` gives the gap between Android's own ring vibration and Thrum's (if "Vibrate for calls" is on). If the gap shows the catch-up is off, it gets corrected from that number.

Needs Mutalib: call access unlocked (App info → : → Allow restricted settings), Thrum allowed to "Modify system settings" once, and a few real calls from another phone.

Task 32: Can calls be caught with the Phone permission instead? (now, alongside 31)

In plain words. Today Thrum learns about a call by reading notifications, and that permission is exactly what makes Play Protect block installs and what locks it on your Pixel. Android can also tell an app "the phone is ringing" through the Phone permission, which isn't on Play Protect's list. This task finds out, with evidence, whether that's good enough. Nothing in the real app changes.

Five questions to answer

1. Does the "ringing" signal reach Thrum when it isn't open, after it's swiped away, and after a restart, on Android 17?

2. Is it earlier or later than the call notification (0.3–0.7 s today)?
3. What does Android's permission box say? It may read "make and manage phone calls", which would put people off.
4. Can Thrum keep the vibration going for the whole ring from that signal? Ring mode re-asserts every 2 s today, which needs Thrum running, and Android limits what an app woken in the background may start.
5. Does Play Protect let such an app install from WhatsApp on a friend's phone?

Steps

1. Research: Android's documentation (the `PHONE_STATE` broadcast and the list of broadcasts still delivered to closed apps, `READ_PHONE_STATE`, the background start limits, Play Protect's enhanced fraud protection list) and real people (Stack Overflow, Hacker News, Reddit, XDA, GitHub issues), with dates on everything. Written up in `docs/research/phone-permission-calls.md`.
2. A test-build-only probe (`app/src/debug/`, so the real app never has it): a receiver for the ringing signal that logs its time, beside a log line for the notification's arrival. Developer tools gets a "Phone permission test" button, which shows Android's real permission box (question 3, screenshotted).
3. Five or more real calls, in Ring and Vibrate, screen on and off, Thrum open, closed, swiped away and after a restart. Times read from `adb logcat`.
4. Question 5: a separate tiny test app (its own name, so it can't touch Thrum) with the Phone permission and no notification access, sent to one friend through WhatsApp. Their phone says yes or no.

Done when all five have an answer with its evidence, written up, and Mutalib has what he needs to choose Task 33's path.

Needs Mutalib: real calls, tapping Allow on the permission box, and one friend's phone.

Task 37: "Your phone's ringtones" list (after 31)

In plain words. A section in Music listing the ringtones already on your phone (the Pixel ones, and any you added), so you can feel them, tune them and set them as your ringtone like any song.

The two traps, and the fixes

- The Music scan deletes any scanned song it no longer finds, and it skips ringtones on purpose. A ringtone added to the library would be deleted by the next scan. Fix: `Track.missingAfterScan` only ever removes songs of kind `MUSIC`, with a test proving a ringtone survives a rescan.
- `MyHaptics.isMade` counts anything that isn't `MUSIC` as made by the user, so ringtones would appear in My Haptics. Fix: a new `TrackKind.RINGTONE` that `isMade` leaves out, again with a test. Check first how Room stores the kind, and that R8's enum rule still covers it.

Steps

1. Read the list with Android's own `RingtoneManager` (ringtone type): it includes the phone's built-in ringtones, which live outside the music folders. Leave out Thrum's own clips in Ringtones/Thrum.
2. Store them as tracks of kind `RINGTONE`. Haptics are made the same way as songs ("as I play them" or in the background), and Instant Feel later applies to them too.

3. Music tab: the section with its own heading and rows like the songs, with the same : menu (Set as ringtone, Tune, Share, Export; no Rename, no Delete). Short ringtones are set as they are (step 2 of Task 31).
4. Sacred Rule 4 check: nothing is shipped or copied out; haptics come from audio already on the phone, and an export still has no sound.

Done when: the Pixel's ringtones appear and the count matches Android's own ringtone list; each one plays with its vibration; a music rescan leaves them in place; none appear in My Haptics; Thrum's clips aren't in the list.

Task 34: The sound-to-vibration gap in the player (measuring)

In plain words. It measures how far apart the sound and the vibration are when you play a song in Thrum, in milliseconds, so "feels in time" becomes a number. The phone's own microphone hears the sound and its motion sensor feels the motor.

Steps

1. A click track made on the laptop: sharp clicks at a steady tempo, so every beat has a clear start in both the sound and the vibration.
2. A test-build-only screen in Developer tools that records the microphone (`AudioRecord`, using its own timestamp of when each sound frame was captured) and the motion sensor (its events carry the same clock) while Thrum's player plays the click track and two real songs. `RECORD_AUDIO` lives in the test build's manifest only; nothing is saved off the phone.
3. Find each click in the sound and each kick in the motion, line them up, and report the typical gap and the worst gap, per song. Measure the sensor's real reading rate first instead of assuming it.
4. If the gap is felt, fix it in the player (a measured offset) and measure again.

Done when the typical and worst gaps for three tracks are written in `PLAN.md`, and Mutalib's hand agrees with the numbers.

Needs Mutalib: to allow the microphone once on the test build, and the phone flat on a table.

Task 35: Instant Feel

In plain words. A song you haven't played before starts in about a second and a half instead of making you wait for the whole song to be read. Thrum reads the opening first, starts, and reads the rest while you listen. Measured on 7 October: the first 45 s read in 1.3–1.4 s, and reading runs 10–28 times faster than the music plays.

Steps

1. First check whether the score's strength scaling depends on the whole song. If it does, the opening is played with a provisional score and swapped for the final one at the next piece boundary. Reading the whole song finishes long before the player gets past 45 s, so only the first piece or two use the provisional version. The haptic that gets saved must be exactly what a normal read makes (a test compares them step for step).

2. `Player.kt`: start after the opening, add pieces as they're ready, and never play past what has been read.
3. The 4 October crash was the background maker and the player editing the waiting list at the same time. That scenario is rerun on purpose with the background maker going, and the queue rules are tested on the PC.
4. The words that promise "about 8 seconds" (`make_as_played_help`, and the making messages) are rewritten to the measured truth.

Done when a 3-minute song never played before starts within 2 s on the Pixel (timed with logcat), with no lag his hand can feel, and the saved haptic matches a normal read.

Task 38: The haptic-channel test file

In plain words. Sound To Haptics puts the vibration inside the sound file itself, and your Pixel plays both from one file. This makes one test file like that from AIZO with Thrum's own rhythm inside, so you can feel it against Thrum's current method. It also checks whether a ringtone file with the vibration inside would vibrate in Ring mode with no call feature at all, which could matter for friends. The app doesn't change, and the file stays on the laptop and your phone (it's your song, so it never goes in the repo).

Steps

1. Research the format before building: how Android knows a file carries haptic channels (the `ANDROID_HAPTIC` tag), what the haptic channel's samples have to look like for the motor, and the Pixel's own motor frequency from `dumpsys vibrator_manager`. Read the AOSP source and real people's write-ups, not memory.
2. Build the file on the laptop: AIZO's first 45 s with Thrum's score turned into a haptic channel, using Python and the FFmpeg already installed. Copy it to the phone (Mutalib says where).
3. A small test-build player in Developer tools that plays it with Android's haptic channels switched on (they're off unless asked for). `dumpsys` must show the motor under the audio system's control, as it did for Sound To Haptics.
4. Then set it as the ringtone and take a real call in Ring mode, to see whether Android's ringer plays the vibration inside it.

Done when Mutalib has felt it beside Thrum's player and said which is better, and the ring-mode call result is written down.

Task 33: Getting Thrum to friends (after 32)

In plain words. Depends on Task 32's answer, and you choose.

- **If the Phone permission works:** Thrum switches to it and drops notification access completely, so Play Protect has nothing to block and the lock on your Pixel goes away. This changes `PROFILE.md` §7 (why Thrum watches notifications), so it needs your yes first. Call detection is rebuilt, the onboarding call page and the locked-access help are rewritten, and the same real-call checks as Task 31 are repeated.
- **If it doesn't:** a "friends" version from the same code (a Gradle flavour) with no call feature and no notification access, under its own name so it can sit beside the full app. Music, My Haptics, export and Set

as ringtone still work there. On a call, the phone rings with the song but Thrum can't vibrate; Task 38 shows whether the ringtone file itself could.

Done when a friend installs it from WhatsApp with no block, and it runs.